



Hackathon Approach

NPN GCP

OCTOBER 2026



Amritha -Hackathon Approach

Outcomes



Activities

- 5 Teams with not more than 6 members in each team
- 5 use cases to be considered for Hackathon (1 use case per team)
- Require one mentor from the college for each of the teams. Cognizant will also assign a mentor for each of the teams

Expectation from Participants

- Attend the orientation call
- Connect with mentors and get guidance
- Finalize the approach and data for the use case selected
- Build prototype as per the timeline
- Submit the design as per the timelines communicated, in PPT or video form over an email



Framework

Student Task

- Analyse the **current process** and identify inefficiencies.
- Propose a **solution approach** (AI/automation/process redesign).
- Define the **data, tools, and methods** required.
- Suggest **KPIs/metrics** to evaluate success.
- Highlight **risks, adoption challenges, and change management needs**.
- Reflect on **future improvements** and **key learnings**.

Instructions

- **Read the chosen use case carefully** and understand the problem statement.
- Use the **questionnaire provided** as a guide to structure your response.
- Your analysis should cover:
 - Problem understanding
 - Data & inputs
 - Solution design
 - Implementation & expected outcomes
 - Risks & mitigation strategies
 - Reflection & improvements
- Present your findings in a structured format:
 - Report (4–6 pages) **OR** Presentation (8–10 slides).
 - You may use diagrams, flowcharts, or mock-ups if needed.
- Ensure your solution is **practical, innovative, and tied to Google Cloud Platform**.

Framework

Expected Outcome

By the end of this task, each student/team should deliver:

- A **clear articulation of the problem** in the chosen use case.
- A **proposed solution** that leverages AI/automation/process improvement.
- Identification of **data, systems, and stakeholders** involved.
- A set of **KPIs/metrics** to measure success.
- Consideration of **risks, compliance, and adoption challenges**.
- A **structured, professional deliverable** (report or slides) that demonstrates critical thinking, problem-solving, and creativity.

Evaluation Criteria - Guideline

Post Build stage

- Use case and process-flow understanding
- Proposed architecture and the alternatives considered
- Breadth of sample data
- Architecture considerations
- Performance
- User experience
- Integration options for the solution
- Reusability
- Ease of implementation
- Real-time decision capability
- Monitoring approach
- Presentation quality
- Participation in mentor connects

Expectation of Best Solution

Presence of

1. **Architecture – clear diagram, clean code**
2. **User Interface – a working, demoable screen**
3. **Documentation/Video - a short walkthrough of your approach**
4. **Roadmap – what's built now vs. what's next**
5. **Presentation – problem, demo, impact and limits**

Hackathon Use cases

Use Cases 1

Objective: Create a small app where users can add, edit, complete, and delete tasks. This is a great starter project because it teaches basic CRUD, backend logic, and deployment on a VM.

Implementation Steps:

1. Choose stack: Use Flask or Node.js for backend, HTML/CSS/JavaScript for frontend, and SQLite for a beginner database.
2. Create the VM: Launch one Compute Engine Linux VM, open ports 80 and 443, and connect with SSH.
3. Build the app UI: Make pages for task list, add task form, edit modal, and delete button.
4. Set up backend routes: Create endpoints for create, read, update, and delete operations.
5. Add database: Store tasks in SQLite for simplicity, or Cloud SQL if you want a managed database later.
6. Deploy on VM
Run the app with Gunicorn/Node process manager and keep it running in the background.
7. Test end to end: Check add, edit, complete, delete, and refresh behavior.
8. Polish demo: Add a clean homepage, status badges, and counts so the app looks finished.

Features:

- Due date reminders.
- Tags or categories like work, study, personal.
- Dark mode for the UI.
- Task counter showing total, completed, and pending tasks.
- Activity log showing when tasks were created or updated.
- Responsive design so it works on mobile too.

Use Case 2

Objective:

A cloud-based log monitoring and visualization solution that gathers and analyzes application logs to help development teams track system performance, error patterns, and overall health. A dashboard that tracks cloud resource usage and suggests cost-saving actions. It focuses on efficiency and optimization.

Implementation Steps:

- Capture application logs
- Send logs to cloud service
- Visualise logs in dashboard
- Set alert thresholds.

Features:

- Centralised logging
- Real-time log viewing
- Error alerts

Use Case 3

Digital Student Flashcard App

Objective: Create a web app where users can create decks of digital flashcards, test themselves, and flip the cards to see the answers. This project reinforces CRUD operations, array manipulation, and basic state management.

Implementation Steps

- **Choose stack:** Node.js or Python (Flask) for backend, plain HTML/CSS/JavaScript for frontend, and SQLite for the database.
- **Design the card UI:** Create a clean, centered card component that toggles a CSS "flipped" class when clicked to reveal the back.
- **Build deck management:** Build forms to create a new deck (e.g., "Biology 101") and add questions/answers to it.
- **Set up backend routes:** Create endpoints to fetch cards, add new cards, and delete old ones.
- **Deploy:** Run the application on a single Compute Engine Linux VM, opening the necessary web ports.

Features

- **Flip animation:** A clean CSS transition that physically flips the card over.
- **"Got it right" tracking:** Buttons for "Know it" and "Review again" that sort cards during a study session.
- **Progress bar:** A visual indicator showing how many cards are left in the deck.
- **Randomizer:** A shuffle button to mix up the card order.

Use Case 4

Recipe Box & Shopping List Maker

Objective: Build a personal recipe repository where users can save their favorite meals, view ingredients, and click a button to automatically add those ingredients to a consolidated shopping checklist.

Implementation Steps

- **Set up the database:** Create an SQLite database with two tables: one for recipes and one for the active shopping list.
- **Build the recipe form:** Allow users to input a title, preparation time, and a text area for ingredients (one per line).
- **Write the shopping list logic:** Create a backend script that splits the ingredient text block by line and inserts those items into the shopping list table.
- **Build the dashboard:** Create a side-by-side or two-page layout: one for browsing recipes, and one for checking off shopping list items.

Features

- **Servings scaler:** A dropdown to change servings (e.g., 2x or 4x) that automatically multiplies ingredient quantities.
- **Checkable list:** A shopping list page where clicking an item strikes it through.
- **Category tags:** Filter recipes by "Breakfast," "Dinner," or "Dessert."
- **Clear list button:** A single button to wipe the shopping list clean after a grocery trip.

Use Case 5

Local Event Bulletin Board

Objective: Create a digital community board where users can post local events, meetups, or garage sales with a title, date, location, and brief description.

Implementation Steps

- **Design the board layout:** Create a grid of "cards," where each card represents an upcoming event.
- **Build the post form:** Add a form where users can input the event name, date/time, location, and description.
- **Handle date sorting:** Write the database query to automatically sort events so the one happening next appears at the top left.
- **Implement an expiration check:** Write a simple piece of backend logic that hides or deletes events if their date is older than the current day.

Features

- **"I'm Going" counter:** A button on each card that users can click to increment a public RSVP counter.
- **Category badges:** Color-coded tags like "Sports," "Music," "Food," or "Yard Sale."
- **Search by neighborhood:** A simple text filter to find events happening in a specific area or city.
- **Shareable links:** A unique ID query string for each post so users can link directly to a specific event card.



Thank You